

Digital design verification

Verification Plan for 4-bit ALU (2 Arithmetic, 2 Logic Operations)

Made by:

Hagar Emadeldin Fathy Emara

Submitted to:

Eng/ Mohamed Ahmed El-adawy

ALU diagram and specification

- 4-bit ALU with two 4-bit operands, A and B.
- Reset signal (asynchronous active low).
- 2-bit Opcode selects the operation applied to A and B; decode is combinational, sampled at posedge CLK for the pipelined Result register.
- Opcode = 2'b00: arithmetic ADD: $\text{Result} = A + B + \text{Cin}$.
- Opcode = 2'b01: arithmetic SUB: $\text{Result} = A - B$ (two's complement).
- Opcode = 2'b10: logic AND: $\text{Result} = A \& B$.
- Opcode = 2'b11: logic OR: $\text{Result} = A | B$.
- Cin is sampled only during ADD; ignored for SUB, AND, and OR.
- Carry flag active high only during ADD when the addition produces a carry-out past bit 3.
- Borrow flag active high only during SUB when $A < B$ (unsigned).
- Overflow flag active high only during ADD or SUB when signed overflow occurs, and tied low for AND and OR.

Signals

1. rst: Asynchronous active low; all output signals (Result, Carry, Borrow, Overflow, Zero) must be zero.
2. A, B: 4-bit inputs.
3. Cin: 1-bit carry-in, sampled at posedge CLK, and only meaningful when Opcode == 2'b00.
4. Opcode[1:0]: at posedge CLK, selects the operation:
 - 2'b00: ADD $\text{Result} = A + B + \text{Cin}$, Carry reflects the carry-out bit.
 - 2'b01: SUB $\text{Result} = A - B$, Borrow set when $A < B$.
 - 2'b10: AND $\text{Result} = A \& B$, Carry/Borrow/Overflow forced to 0.
 - 2'b11: OR $\text{Result} = A | B$, Carry/Borrow/Overflow forced to 0.

Opcode truth table

Opcode[1:0]	Operation	Category	Result	Flags affected
2'b00	ADD	Arithmetic	$A + B + \text{Cin}$	Carry, Overflow
2'b01	SUB	Arithmetic	$A - B$	Borrow, Overflow
2'b10	AND	Logic	$A \& B$	(Carry/Borrow/Overflow = 0)
2'b11	OR	Logic	$A B$	(Carry/Borrow/Overflow = 0)

5. Result Checker:
 - If reset asserted: check $\text{Result} = 0$.
 - At posedge CLK, decode Opcode.
 - At negedge clk, check Result matches the expected operation output.
 - If Opcode = 2'b00: check $\text{Result} = (A + B + \text{Cin})$ truncated to 4 bits, and Carry equals the true carry-out bit.
 - If Opcode = 2'b01: check $\text{Result} = (A - B)$ truncated to 4 bits, and Borrow = 1 when $A < B$, else 0.
 - If Opcode == 2'b10: check $\text{Result} = (A \& B)$, and Carry = Borrow = Overflow = 0.

If Opcode == 2'b11: check Result = (A | B), and Carry = Borrow = Overflow = 0.

If A or B is loaded with invalid data (X/Z): check Result remains invalid/unknown regardless of Opcode.

6. Carry checker: active high only during ADD when the addition generates a carry-out after bit 3, and 0 for all other operations.
7. Borrow checker: active high only during SUB when A < B, and 0 for all other operations.
8. Overflow checker: active high only during ADD or SUB when the operands' signs are equal and differ from the sign of Result, low during AND and OR.

Functional coverage

We need to cover all output signals:

- Result: cover all values (0-15) for each operation.
- Opcode: cover all 4 operations (ADD, SUB, AND, OR).
- Carry: cover carry generated and not generated during ADD.
- Borrow: cover borrow generated and not generated during SUB.
- Overflow: cover signed overflow occurring and not occurring, for ADD and SUB independently.
- A, B: cover corner values (4'h0, 4'hF, A = B, A > B, A < B).
- Cin: cover Cin = 0 and Cin = 1 during ADD.
- Invalid data: cover A or B driven to X/Z and propagated to Result.

Sequences

Sequences & Constrained Randomization

Reset sequence: with rst = 0.

Add sequence: with Opcode = 2'b00, random A, B, Cin.

Sub sequence: with Opcode = 2'b01, random A, B.

And sequence: with Opcode = 2'b10, random A, B.

Or sequence: with Opcode = 2'b11, random A, B.

Test scenarios

Scenario 1: Add sequence for random number transaction range. For testing addition, carry-out generation, and signed overflow.

Scenario 2: Subtraction sequence for random number transaction range [5:10]. For testing subtraction, borrow generation, and signed overflow.

Scenario 3: And sequence - Or sequence. For testing logic operations and confirming Carry, Borrow, and Overflow stay low throughout.

Scenario 4: Randomized Opcode sequence (all 4 operations) for at least 20 iterations. For testing operation switching across every operation.

Scenario 5: Reset sequence asserted in the middle of operation, for random Opcode. For testing asynchronous reset regardless of the operation in progress.